SIMATS ENGINEERING

CO2_ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Name	**E.B.ASHWINI**
Register Number	**192511422**

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 50**

Question Count: 5

Code of Conduct

I __E.B.Ashwini/192511422_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: __Ashwini_____

Assessment Tasks

Analytical Problem Solving

1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.
2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.
3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.
4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.
5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Analytical Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 – Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Interpretation	20%	Interprets all technical requirements accurately	Interprets most requirements correctly	Partial interpretation	Misinterprets the problem
Analytical Reasoning	25%	Strong reasoning with clear cause-effect analysis	Good reasoning with minor gaps	Basic analysis only	Weak or no analysis
Method Selection	20%	Chooses highly appropriate virtualization and security concepts	Chooses mostly suitable concepts	Partially suitable concepts	Inappropriate approach
Solution Quality	20%	Provides feasible and technically sound solution	Reasonable solution with minor issues	Basic solution with limitations	Unclear or invalid solution
Presentation of Analysis	15%	Well-structured and technically precise	Generally clear	Somewhat unclear	Difficult to follow

Total Marks Awarded:

1. **Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical application**

A hypervisor is the software layer that lets multiple virtual machines share one physical machine, each believing it has dedicated hardware. There are two kinds, and the difference between them matters a lot when the workload is mission-critical, because the choice touches every other layer of the datacenter performance, security posture, licensing cost, and even disaster recovery strategy.

A Type-1 hypervisor (also called "bare-metal" or "native") installs directly on the physical hardware, with no host operating system underneath it. Examples are VMware ESXi, Microsoft Hyper-V (in its enterprise deployment mode), and Xen. Because it talks to the hardware directly through its own thin kernel, it has very little overhead between the VM and the CPU, memory, and disk. It's essentially purpose-built to do one job manage virtual machines and nothing else runs on the same layer that could introduce instability.

A Type-2 hypervisor (also called "hosted") runs as an application on top of a regular operating system for example VMware Workstation or Oracle VirtualBox running on Windows or Linux. Every instruction from the VM has to pass through the host OS's own scheduler and drivers first, which adds an extra layer of translation and unpredictability, since the host OS is also busy running its own background processes, updates, and other applications.

Performance: Type-1 is clearly better here. Since there's no host OS competing for CPU cycles or scheduling priority, VMs get closer to native hardware speed commonly cited as 2–10% overhead versus native, compared to Type-2 hypervisors which can run 15–30% slower depending on the workload. This matters enormously for things like real-time transaction processing or database-heavy applications, where consistent low latency is part of the service-level agreement (SLA).

Security: Type-1 again has the edge. Its attack surface is smaller because there's no general-purpose OS underneath that could be exploited the

hypervisor itself is a small, purpose-built piece of software with a minimal codebase, which makes it easier to audit and harden. A Type-2 hypervisor inherits every vulnerability of its host OS: unpatched Windows or Linux exploits, malware, or even something as simple as a user accidentally installing a compromised browser extension can put every VM on that host at risk. For mission-critical data think patient records or financial transactions that's an unacceptable risk surface.

Manageability: This is really the only place Type-2 has a practical advantage it's easier to install and manage because you're just running it like any other desktop application, which is genuinely useful for personal testing, developer sandboxes, or quick proof-of-concept work. Type-1 requires more specialized skills (vCenter, hypervisor consoles, dedicated admin knowledge, sometimes certification) but it's built for large-scale management, with tools for live migration (moving a running VM between hosts with zero downtime), clustering, automated failover, and centralized monitoring across hundreds or thousands of VMs capabilities a Type-2 hypervisor simply doesn't offer at enterprise scale.

Justification: For mission-critical applications, Type-1 is the better choice overall. The performance consistency, reduced attack surface, and enterprise-grade management tooling outweigh the steeper learning curve and higher upfront licensing/hardware cost. This is exactly why every major cloud provider AWS, Azure, Google Cloud runs Type-1 hypervisors under the hood; it would be unthinkable for a hyperscale provider to run customer workloads on a hosted hypervisor sitting on top of a general-purpose OS.

2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.

When multiple tenants with very different security and compliance needs share the same physical servers which is basically the whole point of cloud computing's cost efficiency isolation becomes the single most important design goal. Finance workloads may need to meet PCI-DSS compliance, healthcare workloads need HIPAA-level protections, and student portals, while less sensitive, still handle personal data (names, grades, financial aid info) that needs protecting. Virtualization achieves the needed isolation in a few layered ways.

First ,each VM gets its own virtual CPU, virtual memory space, and virtual disk, all carved out and enforced by the hypervisor. Even though the finance VM and the student portal VM might be running on the exact same physical server, the hypervisor makes sure one VM can't directly read another's memory or access its storage this is sometimes called the "isolation boundary" and is the fundamental promise virtualization makes to tenants.

Second, network isolation is layered on top through virtual LANs (VLANs) or software-defined networking (SDN), so traffic from the healthcare workload never mixes with traffic from the student portal on the same virtual switch. Combined with strict access control policies (IAM roles, security groups, micro-segmentation), this creates logical "walls" between tenants even without physically separate hardware. A well-designed system might even place finance and healthcare workloads on entirely separate physical clusters as an extra precaution, reserving shared hardware only for lower-sensitivity workloads like the student portal.

Third, resource isolation (CPU/memory limits, reservations, and shares) stops one VM from starving the others important so a spike in student portal traffic during registration week doesn't degrade performance for the finance system,

and so a runaway process in one tenant's VM can't cause a "noisy neighbor" problem for everyone else on the host.

Fourth, storage isolation matters too logical unit numbers (LUNs) or storage volumes should be assigned per tenant with access-control lists so that even administrators need explicit permission to cross tenant boundaries.

Two major attack surfaces:

1. Hypervisor / VM escape vulnerabilities. If an attacker manages to compromise a lower-security VM (say the student portal, which is probably the most exposed to the public internet and has the least stringent access controls), and there's a flaw in the hypervisor itself, they could potentially "escape" that VM's boundary and gain access to the host or to other VMs on the same hardware including the finance or healthcare data. Historical examples like the "Venom" vulnerability (CVE-2015-3456) in QEMU's virtual floppy controller showed this is a real, exploitable risk class, not just theoretical. This is the single biggest risk in a multi-tenant virtualized environment and is why hypervisor patching cadence is treated as critical-priority in any serious cloud security program.

2. Shared hardware side-channel attacks. Even without breaking the VM boundary directly, workloads sharing the same physical CPU cache, memory bus, or last-level cache can potentially leak information through timing-based side channels (similar in concept to the Meltdown/Spectre vulnerability class disclosed in 2018). An attacker running a VM on the same host could theoretically infer data from a neighboring VM just by observing how long certain memory operations take, without ever "breaking in" in the traditional sense. This is a harder attack to pull off but is especially concerning for regulated data like healthcare records, since it doesn't leave the same forensic trail as a traditional intrusion.

3. Study the VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.

Looking at these numbers together, this host is clearly overloaded not just one resource, but multiple resources at once, which is what's making the symptoms so severe and why the boot-time symptom is so much worse than any single metric would predict on its own.

- CPU at 92% is right at the edge of saturation. Industry guidance generally treats sustained CPU utilization above 80–85% as a warning threshold on a virtualization host, because at that level VMs are constantly waiting for their turn on the CPU scheduler (this wait time is often called "CPU ready time" in VMware environments). High CPU ready time causes noticeable slowdowns across every VM on the host, not just one applications feel sluggish even though nothing has technically "crashed."
- Memory at 88% is close enough to the ceiling that the hypervisor may start relying on memory ballooning (reclaiming memory from VMs that aren't actively using it) or, worse, swapping to disk to free up space for VMs that need it. Both of those are dramatically slower than normal RAM access — disk swap in particular can be 100x+ slower — and will drag down performance across the board, compounding the CPU problem.
- Storage I/O latency of 35 ms is high a healthy datastore for typical enterprise workloads usually sits under 10–15 ms, and latency-sensitive applications like databases often expect under 5 ms. A reading of 35 ms points to the storage subsystem (physical disks, SAN, or the network path to storage) being a bottleneck, likely because too many VMs are hitting the same storage array simultaneously, or the storage tier itself (e.g., spinning disks instead of SSD) simply isn't fast enough for the current load.
- Average VM boot time of 170 seconds is the symptom that ties it all together, and it's the most telling number here. A VM normally boots in well under a minute often 20–40 seconds. A boot this slow means the VM

is fighting for CPU cycles to execute its boot sequence, waiting on memory allocation from an already-strained pool, and stuck behind a congested storage queue for reading its boot disk, all at the same time. Boot time is a good "canary" metric because it stresses all three resources (CPU, memory, storage) simultaneously in a short window.

- Likely root cause: the host is over-provisioned too many VMs are packed onto hardware that doesn't have enough CPU, memory, or storage bandwidth to serve them all simultaneously, a classic case of over-commitment ratio exceeding what the hardware can realistically support.

Recommended corrective actions:

- Use load balancing (e.g., VMware DRS, or an equivalent orchestration tool) to migrate some VMs off this host onto less busy hosts in the cluster ideally automated so this rebalancing happens continuously rather than reactively.
- Upgrade the host's physical CPU and memory, or reduce the number of VMs assigned to it (lower the consolidation ratio).
- Move to faster storage SSD-backed or NVMe datastores or investigate whether the storage network itself (iSCSI, NFS, or Fibre Channel) is the actual congestion point rather than the disks themselves.
- Set resource limits, reservations, and shares per VM so that no single workload can starve the others of CPU or memory during peak periods.
- Monitor going forward with proactive alerting at lower thresholds (e.g., 75–80% CPU/memory) so this kind of saturation gets caught and addressed before it degrades to a 170-second boot time.
- Consider capacity planning reviews on a recurring schedule so growth in VM count is matched with hardware upgrades ahead of time, rather than reactively.

4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.

A traditional datacenter treats compute, storage, and networking as separate physical silos, each typically owned and operated by a different specialist team. Provisioning a new application usually means racking a physical server, manually configuring a SAN for storage, and physically cabling and configuring network switches and firewalls a process that can take days or weeks and requires several different teams to coordinate through change-management tickets, approvals, and scheduled maintenance windows.

A Software Defined Datacenter (SDD) abstracts all three of those layers into software that can be provisioned, reconfigured, and torn down programmatically, usually through a single management console or API. This is where the agility improvement really comes from infrastructure becomes something you can define in configuration files and deploy on demand, sometimes called "infrastructure as code."

Compute virtualization: Instead of racking physical servers for every new workload, VMs (or containers) can be spun up in minutes from a pre-built template or image. Need more compute for a seasonal spike, like course registration week at a university? Just deploy more VMs from the template — no physical procurement, shipping, or racking needed. This also enables auto-scaling, where the system itself adds or removes compute capacity based on real-time demand without a human even being involved.

Storage virtualization: Physical disks across multiple servers are pooled into a single virtual storage layer (software-defined storage, like VMware vSAN or Ceph). Storage capacity can be resized, tiered (moving frequently-accessed "hot" data to fast SSD and archival "cold" data to cheaper storage), or reallocated between applications through software commands, instead of manually reconfiguring physical disk arrays or waiting for a storage team to provision a new LUN.

Network virtualization: Software-defined networking (SDN, e.g. VMware NSX or open standards like OpenFlow) lets administrators define virtual networks, firewalls, load balancers, and routing rules entirely in software. A new application can get its own isolated virtual network, complete with its own firewall rules and load balancing, in minutes, without touching a single physical switch or router. This also makes network policy portable the same virtual network definition can be replicated identically in a disaster-recovery site.

Put together, these three layers mean an entire application environment compute, storage, and network can be defined as code and deployed on demand, then version-controlled, replicated, or torn down just as easily. This is a massive agility improvement over the traditional model: faster provisioning (minutes instead of weeks), easier scaling up or down in response to real demand, simpler disaster recovery (since VMs and their storage/network definitions can be replicated or moved between datacenters almost automatically), and lower operational overhead since one smaller team can manage the whole stack through software instead of needing separate hardware specialists for each layer. It also reduces human error, since repeatable automated deployments are far less error-prone than manual physical configuration performed differently by different engineers each time.

5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

The problem: When an organization uses multiple cloud providers (say, AWS for compute and Azure for identity-heavy applications, or a mix of public cloud and on-premises systems), each provider typically has its own separate identity system its own way of representing users, roles, and permissions. If these systems aren't properly federated, a user authenticated in one cloud isn't automatically recognized in the other. This creates "identity mismatches" duplicate accounts, inconsistent permissions, orphaned accounts that never get deactivated, or in the worst case, security gaps where access isn't properly revoked across all systems when an employee leaves the organization.

Why it fails: Without federation, each cloud provider ends up maintaining its own separate copy of user identity data. This leads to synchronization issues (a password change in one system doesn't propagate to another), conflicting attributes (e.g., a user's role is "admin" in one system but "read-only" in another because the two systems drifted out of sync), and a much larger attack surface, since credentials have to be managed and secured in multiple places instead of one every additional identity store is another potential point of compromise. It also creates a poor user experience, since users need to remember and manage multiple sets of credentials, which often pushes them toward weak or reused passwords.

Proposed federation design:

- **Central Identity Provider (IdP):** Set up a single, authoritative identity provider (e.g., Azure AD/Entra ID, Okta, or Ping Identity) that all cloud providers trust. This becomes the "source of truth" for who a user is and what roles they have, so there is exactly one place where an account is created, modified, or deactivated.
- **Federation protocol (SAML or OIDC):** Use SAML 2.0 or OpenID Connect to let each cloud provider trust authentication tokens issued by the central IdP, instead of maintaining separate logins. When a user

authenticates once with the IdP, they get a signed, time-limited token that each cloud provider can independently verify and trust without needing its own copy of the user's password the password itself never has to be shared with the individual cloud providers at all.

- **Single Sign-On (SSO):** Building on the federation protocol, SSO means a user logs in once and gets access across all federated cloud environments, removing the need for multiple accounts and drastically reducing the chance of mismatched or forgotten credentials. This also makes offboarding trivial: disabling one account in the central IdP immediately cuts off access everywhere.
- **Attribute mapping and role normalization:** Define a consistent mapping of roles and attributes across providers (e.g., "Finance-Admin" in the central IdP maps consistently to the correct permission level in both AWS IAM and Azure RBAC) so that permissions stay consistent no matter which cloud the user is accessing, avoiding the exact mismatch problem described in the scenario.
- **Multi-factor authentication (MFA) at the federation point:** Since the central IdP is now the single gateway to every cloud environment, it becomes the natural place to enforce strong authentication MFA required at the IdP protects every downstream system at once, rather than needing to be configured separately in each cloud.
- **Privacy protections:** Only share the minimum identity attributes each cloud provider actually needs (principle of least privilege for data sharing) rather than passing a user's full profile to every system. Encrypt identity tokens in transit (TLS) and set short token lifetimes with refresh mechanisms, so a stolen token has a limited window of usefulness. Log and audit all cross-provider authentication events centrally, so unusual access patterns across the federation are visible in one place instead of scattered across separate provider logs.

This design solves the original problem because identity now lives in one trusted, central place, and every cloud provider simply verifies tokens against that source instead of maintaining its own inconsistent copy of user identities eliminating the root cause of the mismatch rather than just patching around it.